# SECURE MOBILE ENVIRONMENTAL SENSING SYSTEM

# 1. Introduction:

This project focuses on the design and implementation of a mobile robotic system capable of navigating a defined environment while acquiring critical environmental parameters, specifically temperature and humidity. The system provides a flexible and efficient solution for monitoring large areas by integrating autonomous mobility with real-time sensing.

Remote operation is facilitated via Bluetooth communication, enabling precise navigation in four directions and ensuring comprehensive spatial coverage beyond the limitations of static sensing platforms. An ultrasonic sensor is incorporated to detect obstacles, triggering automatic halting when the robot approaches objects, thereby enhancing operational safety and reliability.
A password-protected security mechanism governs access to both environmental data and control commands, ensuring that only authorized users can operate or retrieve information from the system. Locking and unlocking functions are executed remotely via Bluetooth, offering secure and convenient management of the system.

All measurements and system status indicators are presented on an I2C-enabled LCD display, complemented by synchronized audio feedback corresponding to movement commands and lock/unlock operations, which collectively improve user awareness and interaction with the robot.
The **motivation** for this project arises from the growing need for efficient and comprehensive environmental monitoring over wide areas. Fixed sensing systems are limited in coverage, providing data from specific points only, whereas a mobile robot can collect information from multiple locations within the same environment, yielding more representative and accurate datasets. Integrating mobility with sensing also enables dynamic response to obstacles and environmental changes, enhancing operational safety while improving the robot's perception and situational awareness.

**Problem statement** Traditional fixed environmental sensing systems are inherently limited in coverage, as they can only collect data from specific points and cannot monitor wide or dynamic areas comprehensively. Consequently, the collected information may not fully represent the actual environmental conditions. Additionally, such systems often lack adequate security, leaving data and control commands vulnerable to unauthorized access. Ensuring safe operation in environments with potential obstacles is also a challenge, as the system must prevent collisions while collecting data.

## 1.1 Project Objectives

The primary objectives of this project are:

- To design a mobile robot capable of collecting environmental data, specifically temperature and humidity, across wide areas.

- To implement a password-based security mechanism that restricts access to authorized users only.
- To integrate an ultrasonic distance sensor to detect obstacles and ensure safe navigation.
- To display real-time data and system status on an I2C-enabled LCD screen, providing an interactive user interface.
- To enhance the robot's perception and situational awareness by combining mobility with environmental sensing.

## 1.2 Expected Contributions:

**Technical Contribution**
Development of a practical mobile robot for secure and efficient collection of environmental data over large areas.
**Educational Contribution**
Providing hands-on experience in Arduino programming, Bluetooth control, sensor integration, and understanding the fundamentals of intelligent robotics.
**Research and Practical Applications**
Establishing a foundation for future applications in environmental monitoring, smart cities, and intelligent industrial systems, where accurate and secure data acquisition from large areas is required.
**Innovative Contribution**
Integrating security, mobility, perception, and user interaction into a single system, creating a comprehensive model for intelligent robotic

## 2. Main Content (System Description and Methodology):

## System Overview:

The robot's components illustrate how the project's objectives are translated from concept to practical implementation. The integrated design of sensors, control systems, and display enables accurate and secure environmental data collection, while allowing the robot to interact intelligently with both its surroundings and the user.
The system is based on a mobile robot built around an Arduino Uno, designed to navigate within a defined environment while collecting and displaying environmental data. The overall architecture integrates both hardware and software components to enable secure, autonomous, and interactive operation.
Hardware Components:
Arduino Uno – serves as the central control unit.
Motors – provide movement in four directions (forward, backward, left, right).
Temperature and Humidity Sensor – measures environmental conditions.
Ultrasonic Sensor – detects obstacles to prevent collisions.

LCD with I2C Interface – displays sensor readings and system status.
Bluetooth Module – enables remote control of the robot.
Buzzer – provides audio feedback in response to user commands and system states.
Software Components:
The system is programmed in Arduino C++, managing sensor readings, motor control, and user interactions.
A password-based security mechanism restricts access to sensor data and control commands to authorized users only.
Sensor data is continuously processed in real-time to determine appropriate robot actions and ensure safe navigation.
Sensors and Perception

**The robot employs multiple sensors to enhance its perception and understanding of the environment:**

**Ultrasonic Sensor:**
Detects obstacles in the robot's path and automatically halts movement when an object is within a predefined distance.
Ensures operational safety and prevents collisions.

**Temperature and Humidity Sensor:**
Continuously measures environmental parameters.
Data is displayed on the LCD and can be accessed via Bluetooth.

**Bluetooth Module:**
Allows secure remote control of movement commands as well as locking/unlocking the system.
LCD with I2C Interface:
Presents real-time environmental data and system status, providing immediate feedback to the user.

**Data Processing:**
Sensor readings are continuously monitored and analyzed to determine robot behavior, such as stopping for obstacles or moving in the desired direction.
Information from all sensors is integrated to provide comprehensive situational awareness, enabling the robot to perceive and interact intelligently with its environment.
Sensors Used in the Project

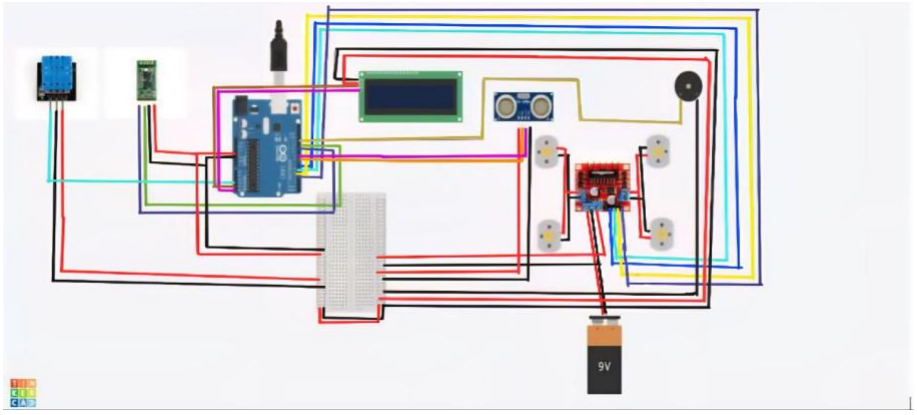
**1. DHT11 Temperature and Humidity Sensor:**
The DHT11 is a widely used digital sensor for measuring temperature and relative humidity (RH). It provides data digitally to the Arduino, with a temperature range of 0–50°C and a relative

humidity range of 20–90% RH, and typical accuracies of ±2°C for temperature and ±5% for humidity. In this project, the DHT11 continuously monitors the environmental conditions within the robot's operational area. The collected data is displayed on an I2C LCD, allowing real-time observation of the environment. This sensor is crucial for achieving the project's goal of wide-area environmental monitoring.(1)

**2. Ultrasonic Distance Sensor (HC-SR04):**

The ultrasonic sensor is used to measure the distance between the robot and surrounding obstacles. It emits high-frequency sound waves and calculates the distance based on the time taken for the waves to reflect back. In this project, the sensor ensures collision-free navigation by automatically stopping the robot when an obstacle is detected within a predefined range. This enhances operational safety and allows the robot to maintain effective mobility while interacting with its environment.(2)

## 2.2Circuit Connection:




The Arduino Uno functions as the main controller of the system, executing the control logic and managing only the direction of the car's movement without speed control by using digital (HIGH/LOW) signals to drive the motors. User commands are received through the Bluetooth module and are executed only after successful password verification using a value stored in the EEPROM.

Once the system is unlocked, movement commands such as forward, backward, turning, or stopping are carried out. Sensor data is continuously monitored, where the ultrasonic sensor detects obstacles and automatically stops the car to ensure safety, while the DHT11 sensor provides temperature and humidity readings displayed on the LCD. Additionally, a buzzer is used to generate audio alerts during events such as incorrect password entry or obstacle detection. This integration ensures secure, safe, and reliable system operation without speed control.

## 2.3 Code Implementation:

This section presents the main parts of the Arduino program used in the project. Each code segment is presented first, followed by a clear and logical explanation describing its purpose and role in the overall system.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>
#include <DHT.h>
#include <EEPROM.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
SoftwareSerial bluetooth(10, 11);
#define DHTPIN A3
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

This section includes all required libraries for hardware communication and data handling. The LCD library enables communication with the I2C display, while the SoftwareSerial library allows Bluetooth communication using digital pins. The DHT library is used to read temperature and humidity values from the DHT11 sensor. The EEPROM library provides non-volatile memory to store the system lock state. Objects are created to represent the LCD, Bluetooth module, and DHT sensor.

```
#define TRIG_PIN 8
#define ECHO_PIN 9
#define BUZZER_PIN 13
#define IN1 7
#define IN2 6
#define IN3 5
#define IN4 4
#define REVERSE_MS 700
const String PASSWORD = "1234";
const int OBSTACLE_DIST = 15;
const int MAX_BEEP_DIST = 50;
const int EEPROM_UNLOCK_ADDR = 0;
```

This section defines all hardware pin connections used in the system, including the ultrasonic sensor, buzzer, and motor driver. Constants are also defined to control system behavior, such as the password used for authentication, the minimum obstacle detection distance, and the EEPROM memory address used to store the lock status. Using constants improves code clarity and simplifies future modifications.

```
bool isUnlocked = false;
bool isMoving = false;
String pwBuffer = "";
char lastCommand = 'S';
```

**These global variables** maintain the current state of the system. The isUnlocked variable determines whether the system accepts movement commands. The isMoving variable indicates whether the robot is currently in motion, enabling obstacle detection only when necessary. The pwBuffer variable temporarily stores the entered password, while lastCommand stores the most recent control command received via Bluetooth.

```
void setup() {
  Serial.begin(9600);
  bluetooth.begin(9600);
  dht.begin();
  lcd.init();
  lcd.backlight();
  lcd.clear();

  pinMode (TRIG_PIN, OUTPUT);
  pinMode (ECHO_PIN, INPUT);
  pinMode (BUZZER_PIN, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
 stopMotors();

 if (EEPROM.read(EEPROM_UNLOCK_ADDR) == 1) {
    isUnlocked = true;
    lcd.setCursor(0, 0);
    lcd.print("Access granted");
    lcd.setCursor(0, 1);
    lcd.print("Ready control");
    tone(BUZZER_PIN, 1000, 200);
    delay(1500);
    lcd.clear();
  } else {
    lcd.setCursor(0, 0);
    lcd.print("Enter password:");
  }
}
```

**The setup() function runs once at system startup**. It initializes serial communication, Bluetooth communication, the LCD, and the DHT sensor. All hardware pins are configured with their correct

input or output modes. The motors are stopped to ensure safe startup. The EEPROM is then checked to determine whether the system was previously unlocked. Based on this value, the system either grants immediate access or prompts the user to enter a password.

**Main Program Logic – loop() Function**

```
void loop() {
  if (!isUnlocked) {
    checkPassword();
    return;
  }
if (bluetooth.available()) {
    char command = bluetooth.read();
    lastCommand = command;
    switch (command) {
      case 'F': moveForward(); break;
      case 'B': moveBackward(); break;
      case 'L': turnLeft(); break;
      case 'R': turnRight(); break;
      case 'S': stopMotors(); break;
      case 'E': lockSystem(); return;
    }
  }

  float h = dht.readHumidity();
  float t = dht.readTemperature();
  static unsigned long lastLCDUpdate = 0;

  if (!isnan(h) && !isnan(t) && millis() - lastLCDUpdate > 1000) {
    lastLCDUpdate = millis();
    lcd.setCursor(0, 0);
    lcd.print("T:");
    lcd.print(t, 1);
    lcd.print("C ");
    lcd.print("H:");
    lcd.print(h, 0);
    lcd.print("%  ");
  }

  if (isMoving) {
    int distance = getDistance();
    if (distance > 0 && distance <= OBSTACLE_DIST) {
      lcd.setCursor(0, 1);
      lcd.print("Obstacle ahead!");
      tone(BUZZER_PIN, 800, 200);
      delay(200);
```

```
      noTone(BUZZER_PIN);
      moveBackward();
      delay(600);
      stopMotors();
    }
  }
}
```

The loop() function controls the system continuously. If the system is locked, it only checks for password input and prevents all other actions. Once unlocked, it listens for Bluetooth commands and executes the corresponding movement functions. Temperature and humidity values are read periodically and displayed on the LCD. When the robot is moving, the ultrasonic sensor checks for obstacles, and if an object is detected within the safe distance, the system warns the user, moves backward, and stops to prevent collision.

**Password Verification Function:**

```
void checkPassword() {
  while (bluetooth.available()) {
    char c = bluetooth.read();
    if (isPrintable(c)) pwBuffer += c;
    lcd.setCursor(0, 1);
    for (int i = 0; i < pwBuffer.length(); i++)
      lcd.print('*');

    if (pwBuffer.length() == PASSWORD.length()) {
      if (pwBuffer == PASSWORD) {
        isUnlocked = true;
        EEPROM.write(EEPROM_UNLOCK_ADDR, 1);
        lcd.clear();
        lcd.print("Access granted");
        tone(BUZZER_PIN, 1000, 200);
        delay(1000);
        lcd.clear();
      } else {
        lcd.clear();
        lcd.print("Wrong password");
        tone(BUZZER_PIN, 300, 200);
        delay(1200);
        lcd.clear();
        lcd.print("Enter password:");
      }
      pwBuffer = "";
    }
  }
}
```

This function handles password entry and validation via Bluetooth. Characters are read and stored temporarily while displaying masked symbols on the LCD. Once the entered password length matches the predefined password, it is validated. A correct password unlocks the system and saves the state in EEPROM, while an incorrect password triggers an error message and sound alert.

**System Lock Function**

```
void lockSystem() {
  isUnlocked = false;
  EEPROM.write(EEPROM_UNLOCK_ADDR, 0);
  stopMotors();
  lcd.clear();
  lcd.print("System locked");
  tone(BUZZER_PIN, 600, 300);
}
```

This function locks the system, stops all motors, updates the EEPROM memory, and provides visual and audio feedback to the user. It ensures that the robot cannot be controlled until the correct password is entered again.

**Distance Measurement Function**

```
int getDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 40000);
  if (duration == 0) return -1;
  return (int)(duration * 0.034 / 2);
}
```

This function measures the distance to nearby objects using the ultrasonic sensor. It calculates the distance based on the time taken for the sound wave to return and converts it into centimeters.

**Motor Control Functions**

```
void moveForward() {
  isMoving = true;
  lcd.setCursor(0, 1);
  lcd.print("Moving forward ");
  digitalWrite(IN1, HIGH);
```

```
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}
void moveBackward() {
  isMoving = true;
  lcd.setCursor(0, 1);
  lcd.print("Moving backward");
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void turnLeft() {
  isMoving = true;
  lcd.setCursor(0, 1);
  lcd.print("Turning left   ");
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}

void turnRight() {
  isMoving = true;
  lcd.setCursor(0, 1);
  lcd.print("Turning right  ");
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

void stopMotors() {
  isMoving = false;
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  lcd.setCursor(0, 1);
  lcd.print("Stopped        ");
}
```

These functions control the robot's movement by setting motor driver pins to appropriate logic levels. Each function updates the LCD to display the current action and sets the movement state accordingly. The stop function safely halts all motors and disables obstacle detection.

```
bool isPrintable(char c) {
  return c >= 32 && c <= 126;
}
```

This function ensures that only valid printable characters are accepted during password entry, preventing unexpected control characters from affecting the system.

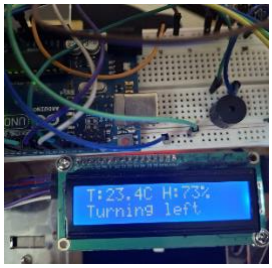
## 3. Results and Discussion:

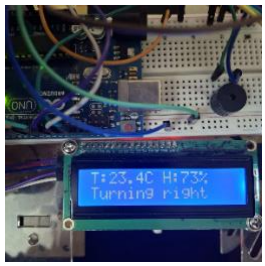
When the car is powered on, the system starts in a secure mode and the user is prompted to enter a password, which is displayed on the LCD as "Enter password". At this stage, the car cannot be controlled and no movement commands are accepted.

If the user enters an incorrect password, the system displays the message "Wrong password" on the LCD and activates an audio warning using the buzzer. The system then requests the password again and remains locked.

When the correct password is entered, the system grants access and displays an access confirmation message. After unlocking, the LCD begins to show real-time temperature and humidity values measured by the DHT11 sensor. The system is now ready to receive movement commands from the Bluetooth module.

The user can control the car in four directions (forward, backward, left, and right) as well as stop the car using Bluetooth commands. The user can also lock the system at any time by sending the command E. When the system is locked, even during movement, the car immediately stops, and the temperature and humidity values are no longer displayed on the LCD.




During operation, if the ultrasonic sensor detects an obstacle in front of the car, the system automatically stops the movement to prevent collision. After stopping, the user can continue controlling the car normally once the path is clear.

Overall, the results show that the system operates correctly by enforcing security, responding accurately to user commands, displaying environmental data, and ensuring safe operation through obstacle detection.

**Discuss system performance, accuracy, limitations, and challenges:**

The system demonstrates stable and reliable performance during operation, responding correctly to user commands after successful password authentication. The sensors provide adequate accuracy for educational purposes, with the ultrasonic sensor detecting obstacles and the DHT11 sensor measuring temperature and humidity. When an obstacle is detected, the system automatically stops the car to ensure safety, demonstrating cognitive behavior based on sensed data. Although the system does not support speed control and relies on sensors with limited precision, it effectively integrates sensing and decision-making to achieve safe and controlled operation.

## 4. Conclusion:

In conclusion, this project presents a comprehensive practical implementation of embedded system concepts by integrating control, sensing, and security into a single smart car system. System reliability is enhanced through the use of EEPROM to store the lock and unlock state, allowing the system to retain its status even after a restart and preventing unintended behavior such as automatic locking or stopping during motion. Logical state variables are also used within the

control logic to manage system states safely and ensure smooth transitions between locked and unlocked modes. Although the system does not include speed control, it successfully meets its educational objectives and demonstrates strong potential for future enhancements, making it a solid foundation for more advanced intelligent and embedded vehicle applications.

## References:


1. Source: John Boxall, “Arduino Workshop: A Hands-On Introduction with 65 Projects,” 2013; Arduino.cc official DHT tutorial.
2. Source: Jeremy Blum, “Exploring Arduino: Tools and Techniques for Engineering Wizardry,” 2013; Arduino.cc official Ping tutorial.